

1. Conceitos Fundamentais de HTTP

1.1. Estrutura de um URI

- **Esquema (scheme):** `http://` ou `https://`
- **Host:** Domínio ou endereço IP (ex: `www.example.com`)
- **Porta:** Opcional, padrão é 80 (HTTP) ou 443 (HTTPS)
- **Path:** Caminho para o recurso (ex: `/products/123`)
- **Query string:** Parâmetros de pesquisa (`?id=123&category=electronics`)
- **Fragment:** Parte opcional que aponta para uma secção da página (`#biography`)

Regra prática: O **path** é tudo aquilo que vem após o host e antes da query string. A query string é tudo aquilo que começa com `?`.

1.2. Métodos HTTP

Método	Safe?	Idempotente?	Uso Típico
GET	✓	✓	Obter representação de um recurso
POST	✗	✗	Criar um novo recurso
PUT	✗	✓	Substituir um recurso existente
DELETE	✗	✓	Remover um recurso
PATCH	✗	✗	Atualizar parcialmente um recurso

Definições:

- **Safe:** Método não altera o estado do servidor.
- **Idempotente:** Repetir a mesma request produz o mesmo resultado.

1.3. Status Codes

Código	Descrição	Uso Típico
200 OK	Sucesso	Pedido processado com sucesso
201 Created	Sucesso	Recurso criado com sucesso
400 Bad Request	Erro do cliente	Parâmetros inválidos ou ausentes
401 Unauthorized	Erro do cliente	Credenciais inválidas ou ausentes
403 Forbidden	Erro do cliente	Cliente não tem permissões
404 Not Found	Erro do cliente	Recurso não existe
500 Internal Server Error	Erro do servidor	Erro não esperado no servidor

Regra prática:

- **4xx:** Erros do cliente (pedido mal formado, sem permissões, etc.)
- **5xx:** Erros do servidor (base de dados inacessível, exceções não tratadas, etc.)

1.4. Headers HTTP

Headers Comuns

Header	Descrição	Direção
Content-Type	Tipo de conteúdo enviado/recebido	Request/Response
Content-Type: application/json	Corpo do pedido/pedido em JSON	Request/Response
Content-Type: text/plain	Corpo do pedido/pedido em texto plano	Request/Response
Content-Type: text/html	Corpo do pedido/pedido em HTML	Request/Response
Content-Type: application/csv	Corpo do pedido/pedido em CSV	Request/Response
Authorization	Token ou credenciais para autenticação	Request
Location	URL para redirecionamento	Response
Set-Cookie	Enviar cookie ao cliente	Response
Cookie	Enviar cookie ao servidor	Request

2. Node.js e Express

2.1. Node.js

- **Ambiente de execução JavaScript** para desenvolvimento de aplicações no servidor.
- **NPM (Node Package Manager):** Gerencia dependências e scripts.
- **Módulos:** Organização de código em ficheiros separados (**require** ou **import**).
- **Assincronia:** Baseado em eventos e callbacks (síncronos/assíncronos).

2.2. Express.js

Framework minimalista para construção de aplicações web em Node.js.

Conceitos-chave

- **Rotas (Routes):** Mapeiam métodos HTTP + paths para handlers.
- **Middleware:** Funções que processam requests/responses.
- **Handlers:** Funções que respondem a pedidos.
- **Request Object (req):** Contém dados do pedido (headers, query, params, body).
- **Response Object (res):** Usado para enviar respostas (status, headers, body).

Exemplo de Rota

```
app.get("/users/:id", (req, res) => {  
  const userId = req.params.id;  
  res.json({ userId, name: "Jorge" });  
});
```

Middleware Básico

```
const logger = (req, res, next) => {  
  console.log(`${req.method} - ${req.path}`);  
  next();  
};  
app.use(logger);
```

3. JavaScript Assíncrono

3.1. Callbacks

```
setTimeout(() => {  
  console.log("Callback executado após 1 segundo");  
}, 1000);
```

3.2. Promises

Objetos que representam operações assíncronas.

Métodos Principais

Método	Descrição	Exemplo
<code>then()</code>	Executa quando a promise é resolvida	<code>.then(res => console.log(res))</code>
<code>catch()</code>	Executa quando a promise é rejeitada	<code>.catch(err => console.error(err))</code>
<code>finally()</code>	Executa independentemente do resultado	<code>.finally(() => console.log("Terminado"))</code>
<code>Promise.all()</code>	Aguarda por múltiplas promises	<code>Promise.all([p1, p2])</code>
<code>Promise.resolve()</code>	Cria uma promise resolvida	<code>Promise.resolve(42)</code>

Método	Descrição	Exemplo
<code>Promise.reject()</code>	Cria uma promise rejeitada	<code>Promise.reject(new Error("Erro"))</code>

Exemplo de Promise

```
function delay(ms) {  
    return new Promise(resolve => setTimeout(resolve, ms));  
}  
  
delay(1000).then(() => console.log("Passou 1 segundo"));
```

3.3. Async/Await

Sintaxe para lidar com promises de forma síncrona.

```
async function getData() {  
    const res = await fetch("https://api.example.com/data");  
    const data = await res.json();  
    console.log(data);  
}
```

Regras:

- `async` marca uma função como assíncrona.
- `await` pausa a execução até a promise ser resolvida.

4. Fetch API

Interface nativa do browser para fazer pedidos HTTP.

Método Básico

```
fetch("https://api.example.com/data")  
    .then(res => res.json())  
    .then(data => console.log(data))  
    .catch(err => console.error(err));
```

Com Headers e Method

```
fetch("https://api.example.com/data", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Jorge" })
});
```

5. Middleware em Express

5.1. Tipos de Middleware

- **Aplicação:** `app.use(middleware)`
- **Rota:** `app.get("/path", middleware, handler)`
- **Error Handling:** `(err, req, res, next) => { ... }`

5.2. Exemplo de Middleware

```
const counter = (req, res, next) => {
  req.counter = (req.counter || 0) + 1;
  next();
};
app.use(counter);
```

6. Estrutura de Módulos e Boas Práticas

6.1. Separação de Responsabilidades

Camada	Responsabilidade	Módulos Típicos
API	Recebe request/response	<code>app.js</code> , <code>server.js</code>
Serviço	Lógica de negócio	<code>service.js</code> , <code>services/</code>
Dados	Acesso a base de dados	<code>data.js</code> , <code>db.js</code>
Middleware	Processamento de pedidos	<code>middleware.js</code> , <code>loggers/</code>

6.2. Validações

- Validar dados de entrada (`body`, `params`, `query`).
- Usar middlewares para validar automaticamente.

Exemplo:

```
const validateBody = (schema) => (req, res, next) => {  
  const { error } = schema.validate(req.body);  
  if (error) return res.status(400).json({ error: error.message });  
  next();  
};
```

7. Headers e Autenticação

7.1. Autenticação com Tokens

- **Header `Authorization`:** Envia token no formato `Bearer <token>`.
- **Armazenamento:** Tokens são armazenados no cliente (localStorage, cookies).

Exemplo:

```
app.use((req, res, next) => {  
  const authHeader = req.headers.authorization;  
  if (!authHeader) return res.status(401).send("Unauthorized");  
  const token = authHeader.split(" ")[1];  
  // Validar token...  
  next();  
});
```

8. Validações e Tratamento de Erros

8.1. Validações de Dados

- Validar existência de propriedades.
- Validar formatos (ex: email, número).
- Validar valores (ex: `length > 0`).

Exemplo com Joi (biblioteca de validação):

```
import Joi from "joi";  
  
const schema = Joi.object({  
  name: Joi.string().required(),  
  age: Joi.number().min(18)  
});  
  
const validate = (req, res, next) => {  
  const { error } = schema.validate(req.body);  
  if (error) return res.status(400).json({ error: error.message });  
};
```

```
    next();  
  };
```

8.2. Tratamento de Erros

- Usar `try/catch` em funções assíncronas.
- Middlewares de erro para capturar erros.

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send("Algo correu mal!");  
});
```

9. Conceitos Avançados de JavaScript

9.1. Closures

Função que captura o ambiente léxico em que foi definida.

```
function outer() {  
  const x = 10;  
  return function inner() {  
    console.log(x); // Acessa x do ambiente de outer  
  };  
}  
const innerFunc = outer();  
innerFunc(); // Output: 10
```

9.2. `this` em Funções

Contexto	Valor de <code>this</code>
Função global	<code>window</code> (browser) ou <code>global</code> (Node.js)
Método de objeto	Objeto que chamou o método
Arrow function	Herda <code>this</code> do contexto exterior
Chamada via <code>call/apply</code>	Objeto passado como argumento

10. Arrays e Métodos Funcionais

10.1. Métodos Principais

Método	Descrição	Exemplo
<code>map()</code>	Transforma cada elemento	<code>[1,2,3].map(x => x*2) → [2,4,6]</code>
<code>filter()</code>	Filtra elementos	<code>[1,2,3].filter(x => x > 1) → [2,3]</code>
<code>forEach()</code>	Itera sobre elementos	<code>[1,2,3].forEach(x => console.log(x))</code>
<code>reduce()</code>	Reduz array a um valor	<code>[1,2,3].reduce((a,b) => a+b) → 6</code>

11. Templates e Handlebars

11.1. Handlebars

Motor de templates para renderizar HTML dinamicamente.

Exemplo de Uso

```
<!-- template.hbs -->
<ul>
  {{#each users}}
    <li>{{name}} ({{email}})</li>
  {{/each}}
</ul>
```

```
res.render("template", { users: [{ name: "Jorge", email: "jorge@isel.pt" }] });
```

11.2. Expressões Comuns

Expressão	Descrição
<code>{{name}}</code>	Acessa propriedade <code>name</code> do contexto
<code>{{#if condition}}</code>	Bloco condicional
<code>{{#each items}}</code>	Itera sobre array
<code>{{#unless condition}}</code>	Inverso de <code>if</code>
<code>{{> partial}}</code>	Inclui template parcial

Termos-Chave

URI, scheme, host, path, query string, fragment, GET, POST, PUT, DELETE, PATCH, safe, idempotente, status code, 200, 201, 400, 401, 403, 404, 500, Content-Type, Authorization, Location, Set-Cookie, Cookie, Node.js, NPM, Express, middleware, handler, Request, Response, callback, Promise, then(), catch(), finally(), Promise.all(), async, await, fetch, Headers, Bearer token, closure, this, map(), filter(), forEach(), reduce(), Handlebars, template, partial, validation, error handling, Joi, separation of concerns, modules, service layer, data access layer.

Observações Finais:

- **Temas recorrentes:** HTTP, Express, JavaScript assíncrono e validações são os pilares da disciplina.
- **Evolução:** Do básico (HTTP, Node.js) para o avançado (middlewares, estruturas de módulos, autenticação).
- **[VERIFICAR]:** Alguns conceitos de closure e `this` não foram aprofundados nos testes, mas são fundamentais para JavaScript.
- **[ORDEM INCERTA]:** Tópicos como Handlebars e estruturas de módulos apareceram em anos diferentes, mas são transversais.